Learning Objectives

Learners will be able to…

- **Index user-defined classes**
- **Cache important information with hash tables**
- **Securely salt and hash passwords**
- **Balance loads with consistent hashing**

Real-World Applications of Hash Tables

Practical Applications of Hash Tables

This assignment is designed to elevate your understanding of hash tables from foundational knowledge to a more nuanced and comprehensive level, focusing on real-world applications, performance optimization, and best practices. You'll learn how hash tables are implemented in complex systems, how to optimize their performance, and how to use them effectively in concurrent environments.

Database Indexing

Hash tables are fundamental in the design of database indexing systems. They enable quick data retrieval, a crucial requirement for any database system.

- **How it Works**: Hash tables map keys to specific locations in the database, allowing for fast access to data.
- **Benefits**: Significant reduction in the time required to find records, update data, and manage large datasets.

Caching Mechanisms

Caching is a technique used to store frequently accessed data in a temporary storage area, making data retrieval faster and more efficient. Hash tables are often used in implementing cache algorithms like LRU (Least Recently Used) and LFU (Least Frequently Used).

- **How it Works**: Hash tables provide constant-time complexity for insertions, deletions, and data retrieval, making them ideal for caching where fast access is crucial.
- **Benefits**: Improved application performance and reduced latency by minimizing the need to fetch data from slower storage systems.

Cryptographic Applications

Hash tables, in combination with cryptographic hash functions, are widely used in various security applications, including hashing passwords for secure storage or ensuring data integrity with hash checks.

- **How it Works**: Cryptographic hash functions generate a unique hash for data or passwords. The hash is stored and used for verification, without exposing the original data or password.
- **Benefits**: Enhanced security and data integrity, with fast verification

times.

Load Balancing in Distributed Systems

Consistent hashing, a variant of hashing, is often used in distributed systems for load balancing. It ensures that a change in the number of servers does not significantly change the mapping of keys to servers.

- **How it Works**: Consistent hashing distributes keys uniformly across servers. When a server is added or removed, only a small portion of keys need to be remapped.
- **Benefits**: Improved distribution of load and better handling of server addition/removal without major reconfiguration.

The versatility of hash tables makes them an indispensable tool in computer science and software development. From speeding up database operations to securing data and balancing loads in distributed systems, hash tables find utility in numerous critical applications. Understanding these real-world applications not only broadens your perspective on where and how hash tables are used but also provides insights into designing solutions for complex problems using this powerful data structure. As we proceed, we'll dive deeper into each of these applications, dissecting their mechanisms and learning how to implement them effectively.

Implementing Data Indexing Using Hash Tables

Indexing Data

In this section, we'll explore how hash tables are used for data indexing by providing a practical coding example. Data indexing is a method to quickly access data in a database, and hash tables make this process efficient, especially for operations like search, insert, and delete. We'll create a simple indexing system for a dataset of users, where each user has a unique ID and associated information.

Assume we have a dataset of users, each with a unique ID and associated information like name and email. We want to create an index that allows us to quickly access user information based on the ID.

Implementation

We'll implement a simple hash table in C++ to serve as our indexing structure. This example will focus on the indexing aspect; in a real-world scenario, you'd likely interface with a database or another form of persistent storage.

Start by creating the `User` class. This class holds information like `id`, `name`, and `email` for a hypothetical user. Define a constructor that takes `string` values for the `id`, `name`, and `email` attributes.

```
class User {
public:
    std::string id;
    std::string name;
    std::string email;

    User(std::string id, std::string name, std::string email)
        : id(id), name(name), email(email) {}
};
```

To help us better understand the state of the program, we are going to override the `<<` operator. This operator will allow us to print the `User` object in a human-readable way, indicating that this is an object of type `User` and showing the attribute names along with their values. Finally, enclose each `User` object with curly brackets.

```
    // Override << operator for easy display of user information
    friend std::ostream& operator<<(std::ostream& os, const
        User& user) {
        os << "User{id='" << user.id << "', name='" << user.name
        << "', email='" << user.email << "'}";
        return os;
    }
```

Next, we are going to create the `UserIndex` class. This class will be our simple information storage system. In a real-world system, this would most likely be a database or some other persistent storage solution. Again, we are focusing on indexing, so this simple example will suffice.

The `UserIndex` will be an array of pointers to `User` objects. This structure also needs a maximum size. In the constructor, accept an integer representing the maximum size. Then initialize the array with the given size, setting all elements to `nullptr`. We are also going to create a simple hashing function. The `std::string` class has a built-in method called `hashCode`. This converts a string into an integer representation. Since we want the hash code to be a valid index in the array, we are going to perform the modulo operation between the integer representation of the string and the size of the array.

```
class UserIndex {

private:
    User** users; // This will be our simple hash table
    int maxSize;

    int hashFunction(const std::string& userId) {
        int hash = 0;
        for (char c : userId) {
            hash = hash * 31 + c;
        }
        return std::abs(hash) % maxSize;
    }

public:
    UserIndex(int size) : maxSize(size) {
        users = new User*[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            users[i] = nullptr;
        }
    }

    ~UserIndex() {
        for (int i = 0; i < maxSize; ++i) {
            delete users[i]; // Free the allocated memory for
        each user
        }
        delete[] users; // Free the memory allocated for the
        array
    }
};
```

Next, we will create the `addUser` method to insert a `User` object into the hash table. Again, we are keeping things simple, so we are not going to worry about collision resolution strategies right now. However, we will print a message to inform us of a collision. Real-world applications should handle these collisions using previously discussed strategies.

```
    // Method to add user to index
    void addUser(User* user) {
        int index = hashFunction(user->id);
        if (users[index] != nullptr) {
            std::cout << "Collision detected for index: " <<
        index << ". Handling not implemented in this example."
        << std::endl;
            return;
        }
        users[index] = user;
    }
```

The `getUser` method takes the ID of a user, converts it to a hash code, and returns the `User` object associated with the given ID. If we try to access an ID not in the hash table, `nullptr` will be returned.

```
    // Method to retrieve user information from index
    User* getUser(const std::string& userId) {
        int index = hashFunction(userId);
        return users[index];
    }
```

Finally, the `displayIndex` method iterates through the underlying array and prints the `User` objects. If a slot is `nullptr`, it prints “null”.

```
    // Method to display the index (for demonstration purposes)
    void displayIndex() {
        std::cout << "[";
        for (int i = 0; i < maxSize; ++i) {
            if (users[i] != nullptr) {
                std::cout << *users[i];
            } else {
                std::cout << "null";
            }
            if (i != maxSize - 1) {
                std::cout << ", ";
            }
        }
        std::cout << "]" << std::endl;
    }
```

Testing the Code

Create the main method which instantiates a `UserIndex` object with a maximum size of `51`. Remember, it is a good idea to use a prime number for the table size. Add two users to the hash table. Search for the user with the ID `417` and print the user. Lastly, print the state of the hash table.

```
int main() {

    UserIndex userIndex(51);
    // Add users
    userIndex.addUser(new User("417", "John Doe",
        "john@example.com"));
    userIndex.addUser(new User("236", "Jane Smith",
        "jane@example.com"));

    // Retrieve and display user information
    User* retrievedUser = userIndex.getUser("417");
    if (retrievedUser != nullptr) {
        std::cout << "Retrieved User: " << *retrievedUser <<
        std::endl;
    } else {
        std::cout << "User not found!" << std::endl;
    }

    // Displaying the entire index (for demonstration)
    userIndex.displayIndex();

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class User {
public:
    std::string id;
    std::string name;
    std::string email;

    User(std::string id, std::string name, std::string email)
        : id(id), name(name), email(email) {}

    friend std::ostream& operator<<(std::ostream& os, const
        User& user) {
        os << "User{id='" << user.id << "', name='" << user.name
        << "', email='" << user.email << "'}";
        return os;
    }
};

class UserIndex {

private:
    User** users;
    int maxSize;

    int hashFunction(const std::string& userId) {
        int hash = 0;
        for (char c : userId) {
            hash = hash * 31 + c;
        }
```

```
        return std::abs(hash) % maxSize;
    }

public:
    UserIndex(int size) : maxSize(size) {
        users = new User*[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            users[i] = nullptr;
        }
    }

    ~UserIndex() {
        for (int i = 0; i < maxSize; ++i) {
            delete users[i];
        }
        delete[] users;
    }

    void addUser(User* user) {
        int index = hashFunction(user->id);
        if (users[index] != nullptr) {
            std::cout << "Collision detected for index: " <<
        index << ". Handling not implemented in this example."
        << std::endl;
            return;
        }
        users[index] = user;
    }

    User* getUser(const std::string& userId) {
        int index = hashFunction(userId);
        return users[index];
    }

    void displayIndex() {
        std::cout << "[";
        for (int i = 0; i < maxSize; ++i) {
            if (users[i] != nullptr) {
                std::cout << *users[i];
            } else {
                std::cout << "null";
            }
            if (i != maxSize - 1) {
                std::cout << ", ";
            }
        }
        std::cout << "]" << std::endl;
    }
};

int main() {
    UserIndex userIndex(51);
    userIndex.addUser(new User("417", "John Doe",
        "john@example.com"));
    userIndex.addUser(new User("236", "Jane Smith",
        "jane@example.com"));

    User* retrievedUser = userIndex.getUser("417");
```

```
    if (retrievedUser != nullptr) {
        std::cout << "Retrieved User: " << *retrievedUser <<
        std::endl;
    } else {
        std::cout << "User not found!" << std::endl;
    }

    userIndex.displayIndex();

    return 0;
}
```

You should see the following output:

```
Retrieved User: User{id='417', name='John Doe',
        email='john@example.com'}
[null, null, null, null, null, null, null, null, null, null,
        null, User{id='236', name='Jane Smith',
        email='jane@example.com'}, null, null, null, null, null,
        null, null, null, null, null, null, null, null, null,
        null, null, null, null, null, null, null, null, null,
        null, User{id='417', name='John Doe',
        email='john@example.com'}, null, null, null, null, null,
        null, null, null, null, null, null, null, null, null]
```

This example demonstrates the basic concept of using a hash table for data indexing. In practice, you'd need to handle collisions more robustly and possibly deal with issues like dynamic resizing. However, this provides a solid foundation for understanding how hash tables facilitate quick data retrieval in indexing systems. Hash tables are often presented as a collection of key-value pairs. When we think of values as user-defined classes, we see how much more utility a hash table can provide in addition to the performance benefits.

Utilizing Hash Tables in Caching Algorithms

Hash Tables and Caching

Hash tables play a crucial role in implementing efficient caching algorithms, especially the Least Recently Used (LRU) and Least Frequently Used (LFU) caches. These caching strategies are essential for optimizing performance in applications where quick access to the most relevant data is critical. Let's delve into how hash tables facilitate these caching mechanisms.

The idea behind caching is to balance performance and memory usage. A doubly-linked list helps in maintaining the order of cached data. The subset of data in the cache and its ordering depends on the caching policy, like LRU, which aims to maximize hit rate within the given capacity. Consequently, operations like insertion and deletion involve both the hash table and the list.

The data in the hash table can be altered based on user needs. Importantly, the position of a node within the list must be actively maintained. Any form of access necessitates repositioning the node to its correct spot.

Least Recently Used (LRU) Cache

The LRU caching algorithm evicts the least recently accessed item when the cache reaches its capacity. This method is beneficial in scenarios where recently accessed items are more likely to be accessed again.

Implementation Overview:

- **Data Structures**: A hash table for $O(1)$ access time, coupled with a doubly-linked list to maintain the order of elements based on their usage.
- **Operations**:
  - **Access/Update**: When an item is accessed or updated, it's moved to the front (head) of the list, indicating that it's the most recently used.
  - **Insertion**: If the cache is full, the least recently used item (at the tail of the list) is removed. The new item is inserted at the head of the list.
  - **Eviction**: Upon removal, the item is detached from the list, and the corresponding entry is deleted from the hash table.

Least Frequently Used (LFU) Cache

The LFU caching algorithm evicts the item with the lowest usage frequency when the cache reaches its capacity. This strategy is effective when the application benefits from retaining frequently accessed items over a longer period.

Implementation Overview:

- **Data Structures**: A hash table for $O(1)$ access time, combined with a structure to maintain the frequency of each item, typically using a combination of hash tables and doubly-linked lists.
- **Operations**:
  - **Access/Update**: When an item is accessed or updated, its frequency count is incremented. The item's position is adjusted to maintain the order based on frequency.
  - **Insertion**: If the cache is full, the item with the lowest frequency (and the oldest among them if there are multiple) is removed. The new item is inserted with a frequency count of 1.
  - **Eviction**: Like LRU, the least frequently used item is removed from the frequency tracking structure and the hash table.

Implementing a Basic LRU Cache

Let's implement a basic LRU Cache using a hash table and a doubly-linked list. Instead of creating our own hash table from scratch, we will use the standard `unordered_map` from the C++ Standard Library.

Begin by creating the `LRUCache` class. Since we are using a doubly-linked list, we also need a `Node` class with integers for both the key and value. The class also includes pointers for the next and previous nodes in the list. Create a constructor for the `Node` class that takes integers for the key and value.

```
class LRUCache {
    private:
    struct Node {
        int key;
        int value;
        Node* prev;
        Node* next;

        Node(int k, int v) : key(k), value(v), prev(nullptr),
        next(nullptr) {}
    };

};
```

Next, create the constructor for the `LRUCache` class. You need constants for the capacity of the cache, the hash table, and the head and tail nodes in the list. The constructor for the `LRUCache` class takes an integer and sets it as the cache capacity.

```
    private:
    int capacity;
    std::unordered_map<int, Node*> map;
    Node* head;
    Node* tail;

    public:
    LRUCache(int cap) : capacity(cap), head(new Node(0, 0)),
        tail(new Node(0, 0)) {
        head->next = tail;
        tail->prev = head;
    }
```

▼ The head and tail nodes

> The head and tail nodes function differently than in a traditional list. These nodes act as markers only, and they do not contain any useful information. Inserting a node at the beginning of the list means placing it right after the `head` node. Similarly, evicting the last node means removing the node just before the `tail` node.

The public `get` method returns the value of a key in the hash map or `-1` if it is not present. However, since the key-value pair (a node) is being used, we need to move the node to the front of the list. If the key exists, create a node based on the key-value pair. Use the remove method to `remove` the node from the list. Then, use the `insert` method to add the node to the head of the list.

```
    int get(int key) {
        if (map.find(key) != map.end()) {
            Node* node = map[key];
            remove(node);
            insert(node);
            return node->value;
        }
        return -1;
    }
```

The public `put` method performs several actions. It handles adding or overwriting the key-value pair in the map and inserting the key-value pair as a node at the front of the list. If the key is already in the map, it overwrites the information. Remove the node from the list. Then, check if the list is at capacity. If so, evict the last node in the list. Finally, insert a new node with the given key-value pair.

```
    void put(int key, int value) {
        if (map.find(key) != map.end()) {
            remove(map[key]);
        }
        if (map.size() == capacity) {
            remove(tail->prev);
        }
        insert(new Node(key, value));
    }
```

The `remove` method is a private method that handles removing a key-value pair from the map and the list.

```
    private:
    void remove(Node* node) {
        if (node == nullptr) return;

        // Remove from hash map
        map.erase(node->key);

        // Update linked list connections
        if (node->prev) node->prev->next = node->next;
        if (node->next) node->next->prev = node->prev;
    }
```

The `insert` method is a private method that handles inserting a `key-value` pair in the map and a node at the beginning of the list.

```
    void insert(Node* node) {
        // Insert into hash map
        map[node->key] = node;

        // Insert node right after head
        node->next = head->next;
        node->prev = head;

        if (head->next) head->next->prev = node;
        head->next = node;

        // Ensure tail's prev is updated
        if (node->next == tail) {
            tail->prev = node;
        }
    }
```

Testing the Code

We'll create a `printList` method to help us observe the state of the list. This method isn't needed when implementing an `LRU` cache. Note that the head node doesn't contain data, so we'll create a variable representing the

current node, starting at `head->next`. Iterate over the list, printing the `key-value` pairs.

```
    public:
    void printList() {
        Node* current = head->next;
        while (current != tail) {
            std::cout << "Key: " << current->key << " Value: "
        << current->value << std::endl;
            current = current->next;
        }
    }
```

Add the `main` method. We will keep this simple. Instantiate an `LRUCache` object and set its capacity to `3`. Insert three `key-value` pairs. With each insertion, the new node should go to the front of the list. Then, test the `get` method, which should move the node with key `1` to the front of the list. Finally, add a fourth node, which should add the node at the beginning and evict the last node.

```
int main() {
    LRUCache lru(3);

    std::cout << "---Test Put---" << std::endl;
    lru.put(1, 10);
    lru.put(2, 20);
    lru.put(3, 30);
    lru.printList();
    std::cout << std::endl;

    std::cout << "---Test Get---" << std::endl;
    lru.get(1);
    lru.printList();
    std::cout << std::endl;

    std::cout << "---Test Evict---" << std::endl;
    lru.put(4, 40);
    lru.printList();

    return 0;
};
```

▼ Code

> Your code should look like this:
>
> ```
> class LRUCache {
>     private:
>     struct Node {
>         int key;
>         int value;
>         Node* prev;
>         Node* next;
> ```

```
    Node(int k, int v) : key(k), value(v),
    prev(nullptr), next(nullptr) {}
};

private:
int capacity;
std::unordered_map<int, Node*> map;
Node* head;
Node* tail;

public:
LRUCache(int cap) : capacity(cap) {
    head = new Node(0, 0);
    tail = new Node(0, 0);
    head->next = tail;
    tail->prev = head;
}

int get(int key) {
    if (map.find(key) != map.end()) {
        Node* node = map[key];
        remove(node);
        insert(node);
        return node->value;
    }
    return -1;
}

void put(int key, int value) {
    if (map.find(key) != map.end()) {
        remove(map[key]);
    }
    if (map.size() == capacity) {
        remove(tail->prev);
    }
    insert(new Node(key, value));
}

private:
void remove(Node* node) {
    if (node == nullptr) return;

    // Remove from hash map
    map.erase(node->key);

    // Update linked list connections
    if (node->prev) node->prev->next = node->next;
    if (node->next) node->next->prev = node->prev;
}

void insert(Node* node) {
    // Insert into hash map
    map[node->key] = node;

    // Insert node right after head
    node->next = head->next;
    node->prev = head;
```

```
        if (head->next) head->next->prev = node;
        head->next = node;

        // Ensure tail's prev is updated
        if (node->next == tail) {
            tail->prev = node;
        }
    }

    public:
    void printList() {
        Node* current = head->next;
        while (current != tail) {
            std::cout << "Key: " << current->key << "
        Value: " << current->value << std::endl;
            current = current->next;
        }
    }

};

int main() {
    LRUCache lru(3);

    std::cout << "---Test Put---" << std::endl;
    lru.put(1, 10);
    lru.put(2, 20);
    lru.put(3, 30);
    lru.printList();
    std::cout << std::endl;

    std::cout << "---Test Get---" << std::endl;
    lru.get(1);
    lru.printList();
    std::cout << std::endl;

    std::cout << "---Test Evict---" << std::endl;
    lru.put(4, 40);
    lru.printList();

    return 0;
};
```

You should see the following output:

```
---Test Put---
Key: 3 Value: 30
Key: 2 Value: 20
Key: 1 Value: 10

---Test Get---
Key: 1 Value: 10
Key: 3 Value: 30
Key: 2 Value: 20

---Test Evict---
Key: 4 Value: 40
Key: 1 Value: 10
Key: 3 Value: 30
```

The key thing to remember about this implementation is that the head node is not a node in the traditional sense. It does not store any data. Instead, it is a “marker” node used to make operations like insertion and deletion easier.

We’ve covered the fundamental aspects of hash tables and their application in LRU caching. The concepts and code presented can be adapted to more complex caching scenarios, including LFU caches, by refining the data structures and algorithms involved. Hash tables, with their efficient lookups, serve as the backbone for such cache implementations, ensuring that data retrieval remains optimal even as the system scales.

Load Balancing with Consistent Hashing

Hash Tables and Distributing Data

In traditional hashing, we have seen how hash tables need to be resized once the load factor crosses a certain threshold. That means all the key-value pairs have to be rehashed to optimize the distribution. The process can take a lot of time and resources when the hash tables are large. The use of consistent hashing can help minimize this problem.

Consistent Hashing

Consistent hashing is a specialized form of hashing that is particularly useful in distributed systems for load balancing and data distribution. Hash tables like this are often referred to as distributed hash tables. It minimizes the need for rehashing when a hash table is resized, especially in scenarios like distributed caching or sharing databases. This section will discuss the concept of consistent hashing and how to implement it effectively.

Key Concepts:

- **Uniform Distribution**: Data is uniformly distributed across all nodes, minimizing the load imbalance.
- **Minimal Movement**: When a node is added or removed, only a small portion of data is moved to or from the node, reducing the overhead.

Consistent hashing works by mapping keys to nodes in a “ring”. Nodes are assigned a position in the ring. To add a key-value pair, first hash the key. The value is assigned to the first node clockwise from its hash position. Instead of having to rehash the entire table, you only have to rehash the keys in a node once that node has to be resized. The other nodes remain unchanged.

Use Cases for Consistent Hashing

Consistent hashing is widely used in:

- **Distributed Caching**: To distribute the cache data across multiple nodes and minimize re-distribution when nodes are added or removed.
- **Distributed Databases**: To manage data shards across different database instances or servers.

Implementing Consistent Hashing

Here's a simple illustration of implementing consistent hashing. In this example, a `ConsistentHashRing` class is created, simulating the addition and removal of nodes and the assignment of keys to nodes.

Start by creating the `ConsistentHashRing` class. The ring attribute stores nodes and their hashed values. The ring is a `std::map`, which maintains sorted order. The `numberOfReplicas` attribute controls how many virtual nodes are created for each physical node. This improves load distribution. The constructor takes an integer representing the number of replicas.

```
class ConsistentHashRing {
    private:
        std::map<size_t, std::string> ring;
        int numberOfReplicas;

    public:
        ConsistentHashRing(int numberOfReplicas) :
        numberOfReplicas(numberOfReplicas) {}

        size_t hashFunction(const std::string& str) {
            size_t hash = 0;
            for (char c : str) {
                hash = (hash * 31) + c;
            }
            return hash;
        }
};
```

▼ **What are `std::map` and `std::multimap`**

> `std::map` is a container that stores elements in a key-value pair format and allows for sorting based on keys. The `std::multimap` is similar but allows multiple entries with the same key. The `std::map` data structure allows for ordered access. However, it does this at the expense of taking a bit longer to perform operations and taking up a bit more memory.

The `addNode` method is a public method that takes a string representing a node and adds a node to the ring. The loop creates as many virtual nodes as dictated by `numberOfReplicas`. For each virtual node, a number is appended to the node name. This is used to generate a hash code.

```
    void addNode(const std::string &node) {
        for (int i = 0; i < numberOfReplicas; i++) {
        ring[hashFunction(node + std::to_string(i))] = node;
        }
    }
```

The `removeNode` method is a public method that takes a string representing a node and removes it from the ring. This means you have to remove the virtual nodes. Create another loop that runs `numberOfReplicas` times. For each iteration, generate the correct key by adding `i` to the node name and hashing. Use this hash code with the erase method to remove each virtual node.

```
    void removeNode(const std::string &node) {
        for (int i = 0; i < numberOfReplicas; i++) {
            ring.erase(hashFunction(node + std::to_string(i)));
        }
    }
```

The `getNodeForKey` method is a public method that takes a string representing a key. The method returns the name of the node where the key should be found. If the ring is empty, return ““. Otherwise, generate a hash value for the given key. If the key is found in the ring, return that node. If the ring does not contain that exact hash code, we need to calculate the hash code for the nearest node for the given key. Use the `upper_bound(hash)` method call to find the first element that is greater than the hash. If the iterator is at the end, return the first key in the ring.

```
    std::string getNodeForKey(const std::string &key) {
        if (ring.empty()) {
            return "";
        }

        size_t hash = hashFunction(key);

        auto it = ring.upper_bound(hash);
        if (it == ring.end()) {
            it = ring.begin();  // Wrap around to the beginning
        of the ring
        }

        return it->second;
    }
```

Testing the Code

Create the main function that instantiates a `ConsistentHashRing` object with 3 replicas. Add three nodes to the ring. Calculate the node that would be used for the key “Key1” and print it. This should return “Node1”. Now, remove node “Node1” and calculate the node for the key “Key1” once more. This time, it should return “Node2”.

```
int main() {
    ConsistentHashRing hashRing(3);

    hashRing.addNode("Node1");
    hashRing.addNode("Node2");
    hashRing.addNode("Node3");

    std::string nodeForKey = hashRing.getNodeForKey("Key1");
    std::cout << "Node for Key1: " << nodeForKey << std::endl;

    hashRing.removeNode("Node1");

    nodeForKey = hashRing.getNodeForKey("Key1");
    std::cout << "Node for Key1 after removing Node1: " <<
        nodeForKey << std::endl;

    return 0;

}
```

▼ **Code**

Your code should look like this:

```
class ConsistentHashRing {
    private:
        std::map<size_t, std::string> ring;
        int numberOfReplicas;

    public:
        ConsistentHashRing(int numberOfReplicas) :
        numberOfReplicas(numberOfReplicas) {}

        size_t hashFunction(const std::string& str) {
            size_t hash = 0;
            for (char c : str) {
                hash = (hash * 31) + c;
            }
            return hash;
        }

        void addNode(const std::string &node) {
            for (int i = 0; i < numberOfReplicas; i++) {
                ring[hashFunction(node +
        std::to_string(i))] = node;
            }
        }

        void removeNode(const std::string &node) {
            for (int i = 0; i < numberOfReplicas; i++) {
                ring.erase(hashFunction(node +
        std::to_string(i)));
            }
        }

        std::string getNodeForKey(const std::string &key) {
            if (ring.empty()) {
```

```
                return "";
            }

            size_t hash = hashFunction(key);

            auto it = ring.upper_bound(hash);
            if (it == ring.end()) {
                it = ring.begin();  // Wrap around to the
        beginning of the ring
            }

            return it->second;
        }
};

int main() {
    ConsistentHashRing hashRing(3);

    hashRing.addNode("Node1");
    hashRing.addNode("Node2");
    hashRing.addNode("Node3");

    std::string nodeForKey =
        hashRing.getNodeForKey("Key1");
    std::cout << "Node for Key1: " << nodeForKey <<
        std::endl;

    hashRing.removeNode("Node1");

    nodeForKey = hashRing.getNodeForKey("Key1");
    std::cout << "Node for Key1 after removing Node1: " <<
        nodeForKey << std::endl;

    return 0;

}
```

You should see the following output:

```
Node for Key1: Node1
Node for Key1 after removing Node1: Node2
```

Consistent hashing is a fundamental technique in designing scalable, distributed systems. It ensures effective load balancing and minimizes data movement during scaling operations. This concept is particularly crucial in environments with high availability and scalability requirements, such as cloud computing and large-scale web applications. Understanding and implementing consistent hashing allows for the efficient distribution of data across multiple nodes, optimizing system performance and reliability.

Cryptographic Applications of Hash Tables

Hash Tables and Cryptography

info

OpenSSL

The code on this page uses the OpenSSL library. This is not a part of C++'s standard library.

Hash tables, combined with cryptographic hash functions, play a crucial role in various security applications. Cryptographic hash functions are designed to take an input (or 'message') and return a fixed-size string of bytes. The output is typically a 'digest' that is unique to each unique input message. This section explores how hash tables are used in conjunction with cryptographic hash functions to enhance data security, ensure data integrity, and manage passwords securely.

Ensuring Data Integrity with Hash Checks

One of the primary uses of cryptographic hash functions in conjunction with hash tables is to ensure data integrity. This is crucial in various applications, from software distribution to secure communications.

Implementation Overview:

- **File Integrity**: Software distribution systems often provide a hash value along with the download links. After downloading the file, a user can compute the hash value of the downloaded file and compare it with the provided hash value to ensure that the file has not been tampered with.
- **Data Transmission**: In data transmission, sending the hash value of the data along with the data itself allows the receiver to verify the integrity of the received data by comparing the computed hash value of the received data with the sent hash value.

Hash Tables in Password Management

Storing passwords securely is a critical aspect of protecting user data. Hash tables, when used with cryptographic hash functions, provide a secure way to manage passwords.

Implementation Overview:

- **Password Storage**: Instead of storing the actual passwords, systems store the hash value of the passwords. When a user enters a password, the system hashes it and compares it to the stored hash value.
- **Salting**: To prevent attacks using precomputed dictionaries (called rainbow tables), systems add a unique salt to each password before hashing it. The salt and the hash value are both stored in the hash table.

Blockchain and Cryptographic Hash Functions

Blockchain technology heavily relies on cryptographic hash functions. Hash tables are used to quickly access and manage the data in individual blocks.

Implementation Overview:

- **Immutable Ledger**: Each block contains a hash of the previous block, creating a chain. The integrity of the chain relies on these hashes; altering any block would require recalculating all subsequent hashes, which is computationally infeasible.
- **Efficient Verification**: Hash tables allow for efficient and quick verification of block data and the blockchain's integrity.

Using Hash Tables for Password Storage with Salting

Below is a simplistic example demonstrating how a hash table can be used for storing and verifying passwords securely using salting and hashing in C++. This example uses the `std::unordered_map` for the hash table and the OpenSSL library for cryptographic operations.

The `SecurePasswordStorage` class uses hash tables and cryptographic functions to store and verify passwords. We also need to define the `UserCredentials` class which has two string attributes – `salt` and `hashedPassword`. The constructor accepts two string arguments and sets them to their respective attribute.

```
class SecurePasswordStorage {
private:
    struct UserCredentials {
        std::string salt;
        std::string hashedPassword;

        UserCredentials() = default;

        UserCredentials(const std::string &salt, const
        std::string &hashedPassword)
            : salt(salt), hashedPassword(hashedPassword) {}
    };

    std::unordered_map<std::string, UserCredentials> userTable;
```

The `storeUserPassword` method is a public method that takes a user name (string) and its associated password (string). It creates a salted and hashed version of the password. The user name and hashed password are then stored in the map. This ensures a malicious actor cannot access the plain text (not hashed) password.

```
public:
    void storeUserPassword(const std::string &username, const
        std::string &password) {
        std::string salt = generateSalt();
        std::string hashedPassword = hashPassword(password,
        salt);
        userTable[username] = UserCredentials(salt,
        hashedPassword);
    }
```

The `verifyPassword` method is a public method that takes a user name and its associated password. If the user name is not already in the map, return `false`. Otherwise, get the key-value pair associated with the user name from the map. If you hash and salt the given password and get the same result as the value stored in the map, then the given password is correct and the method returns `true`. If they do not match, return `false`.

```
    bool verifyPassword(const std::string &username, const
        std::string &passwordToVerify) {
        auto it = userTable.find(username);
        if (it == userTable.end()) {
            return false;
        }
        const UserCredentials &credentials = it->second;
        std::string hashedPassword =
        hashPassword(passwordToVerify, credentials.salt);
        return hashedPassword == credentials.hashedPassword;
    }
```

The `generateSalt` method is a private method that does not take any parameters. Start by creating a buffer of 16 bytes (`unsigned char` is commonly used for cryptographic operations). `RAND_bytes` is a part of the

OpenSSL library. Fill the `salt` buffer with cryptographically secure bytes. This process returns a `1` if it succeeds. If filling the `salt` buffer was not successful, throw a `std::runtime_error`. Use the `bytesToHex` method to convert the binary `salt` into a hexadecimal string. Return this string.

```
private:
    std::string generateSalt() {
        unsigned char salt[16];
        if (RAND_bytes(salt, sizeof(salt)) != 1) {
            throw std::runtime_error("Failed to generate random
        bytes for salt");
        }
        return bytesToHex(salt, sizeof(salt));
    }
```

The `hashPassword` method takes two strings and returns the hashed password (a string). Start by creating a buffer for the hashed output. Add the salt to the beginning of the password. Use the SHA-256 secure hashing algorithm to hash the salted password. Convert the hashed password to a hexadecimal string and return this string.

```
    std::string hashPassword(const std::string &password, const
        std::string &salt) {
        unsigned char hash[SHA256_DIGEST_LENGTH];
        std::string saltedPassword = salt + password;
        SHA256(reinterpret_cast<const unsigned char*>
        (saltedPassword.c_str()), saltedPassword.size(), hash);
        return bytesToHex(hash, SHA256_DIGEST_LENGTH);
    }
```

The `bytesToHex` method takes an array of signed characters and the length of the array. The method returns a string. Start by setting up an output string stream. This allows us to build the string that will be eventually returned. Iterates over the bytes array, converting each byte into an integer. Once the loop is done, return the string.

```
    std::string bytesToHex(const unsigned char *bytes, size_t
        length) {
        std::ostringstream oss;
        for (size_t i = 0; i < length; ++i) {
            oss << std::hex << std::setw(2) << std::setfill('0')
        << static_cast<int>(bytes[i]);
        }
        return oss.str();
    }

};
```

Testing the Code

The `main` method demonstrates how to use the `SecurePasswordStorage` class to store and verify passwords.

```
int main() {
    SecurePasswordStorage storage;

    storage.storeUserPassword("ExampleUser",
        "CorrectHorseBatteryStaple");

    // Output boolean values
    std::cout << std::boolalpha; // Enable boolean output in
        true/false format
    std::cout << storage.verifyPassword("ExampleUser",
        "CorrectHorseBatteryStaple") << std::endl;
    std::cout << storage.verifyPassword("ExampleUser",
        "HorseBatteryStaple") << std::endl;
    std::cout << storage.verifyPassword("OtherUser",
        "SomeOtherPassword") << std::endl;

    return 0;
}
```

▼ **Code**

> Your code should look like this:
>
> ```
> class SecurePasswordStorage {
> private:
>     struct UserCredentials {
>         std::string salt;
>         std::string hashedPassword;
>
>         UserCredentials() = default;
>
>         UserCredentials(const std::string &salt, const
>         std::string &hashedPassword)
>             : salt(salt), hashedPassword(hashedPassword) {}
>     };
>
>     std::unordered_map<std::string, UserCredentials>
>         userTable;
>
> public:
>     void storeUserPassword(const std::string &username,
>         const std::string &password) {
>         std::string salt = generateSalt();
>         std::string hashedPassword = hashPassword(password,
>         salt);
>         userTable[username] = UserCredentials(salt,
>         hashedPassword);
>     }
>
>     bool verifyPassword(const std::string &username, const
>         std::string &passwordToVerify) {
>         auto it = userTable.find(username);
>         if (it == userTable.end()) {
>             return false;
>         }
> ```

```
        const UserCredentials &credentials = it->second;
        std::string hashedPassword =
        hashPassword(passwordToVerify, credentials.salt);
        return hashedPassword ==
        credentials.hashedPassword;
    }

private:
    std::string generateSalt() {
        unsigned char salt[16];
        if (RAND_bytes(salt, sizeof(salt)) != 1) {
            throw std::runtime_error("Failed to generate
        random bytes for salt");
        }
        return bytesToHex(salt, sizeof(salt));
    }

    std::string hashPassword(const std::string &password,
        const std::string &salt) {
        unsigned char hash[SHA256_DIGEST_LENGTH];
        std::string saltedPassword = salt + password;
        SHA256(reinterpret_cast<const unsigned char*>
        (saltedPassword.c_str()), saltedPassword.size(),
        hash);
        return bytesToHex(hash, SHA256_DIGEST_LENGTH);
    }

    std::string bytesToHex(const unsigned char *bytes,
        size_t length) {
        std::ostringstream oss;
        for (size_t i = 0; i < length; ++i) {
            oss << std::hex << std::setw(2) <<
        std::setfill('0') << static_cast<int>(bytes[i]);
        }
        return oss.str();
    }

};

int main() {
    SecurePasswordStorage storage;

    storage.storeUserPassword("ExampleUser",
        "CorrectHorseBatteryStaple");
    // Output boolean values
    std::cout << std::boolalpha; // Enable boolean output
        in true/false format
    std::cout << storage.verifyPassword("ExampleUser",
        "CorrectHorseBatteryStaple") << std::endl;
    std::cout << storage.verifyPassword("ExampleUser",
        "HorseBatteryStaple") << std::endl;
    std::cout << storage.verifyPassword("OtherUser",
        "SomeOtherPassword") << std::endl;

    return 0;

}
```

You should see the following output:

```
true
false
false
```

Cryptographic hash functions and hash tables are pivotal in ensuring data security and integrity. Whether it's verifying data integrity, securely managing passwords, or maintaining the immutable ledger of a blockchain, the combination of hash tables with cryptographic hash functions forms a cornerstone of modern data security practices. Understanding and implementing these mechanisms is crucial for any system that prioritizes data integrity and security.

Formative Assessment 1

Formative Assessment 2